Image Inpainting and Object Removal

Week 3: Resolved LaMa pipeline, ViTInpainter evaluation and failure analysis

Ivan, Alexander, Egor, Tatiana | Computer Vision 2026 | October 20, 2026

Abstract

We compare classical, CNN-based, Transformer-based and diffusion-based inpainting methods for object removal on the CORNE-Val benchmark. This report covers the third week: diagnosing and fixing the preprocessing issue in pretrained LaMa, integrating the Transformer-based model (ViTInpainter) behind the shared benchmark interface, and evaluating both deep learning approaches quantitatively and qualitatively across all six mask configurations against the OpenCV Telea baseline. We also present an initial failure analysis across three representative failure modes (including shadow-edge hallucinations in both ViTInpainter and LaMa) and explain why the full `FLUX.1-Fill` benchmark run was rescheduled to Week 4 due to high inference time and GPU memory requirements.

Contents

1 New Approaches: Pretrained LaMa and ViTInpainter **2**
- 1.1 Pretrained LaMa (CNN with Fast Fourier Convolutions) . . . 2
- 1.2 ViTInpainter (Vision Transformer Inpainting) . . . 2

2 Motivation **2**

3 Implementation and Differences from Baseline **2**
- 3.1 Resolving the Week 2 LaMa Blurring Issue . . . 3
- 3.2 Main Differences Compared with the Baseline . . . 3

4 Initial Results and Comparison with Baseline **3**
- 4.1 Quantitative Results on CORNE-Val . . . 3
- 4.2 Qualitative Results and Discussion . . . 3

5 Failure Analysis and Problems Observed **6**
- 5.1 Representative Failure Cases . . . 6
- 5.2 Resource Bottleneck with `FLUX.1-Fill` . . . 6

6 Next Steps **7**

7 Code Repository **7**

1 New Approaches: Pretrained LaMa and ViTInpainter

Following the classical OpenCV Telea baseline established in Week 2, this week evaluates two deep learning architectures representing CNN-based and Transformer-based paradigms:

1.1 Pretrained LaMa (CNN with Fast Fourier Convolutions)

LaMa (*Resolution-robust Large Mask Inpainting with Fourier Convolutions*) is a feed-forward CNN architecture built around Fast Fourier Convolutions (FFCs). Unlike standard spatial convolutions whose receptive field grows slowly with depth, FFCs split feature channels into a local spatial branch and a global spectral branch that applies a Real Fast Fourier Transform (Real FFT), convolves in the frequency domain, and transforms back via inverse Real FFT. This grants the network an image-wide receptive field from the very first layers, enabling it to capture periodic background structures (such as pavement, walls, and snow textures) across large missing regions.

1.2 ViTInpainter (Vision Transformer Inpainting)

ViTInpainter replaces spatial and spectral convolutions with a Vision Transformer backbone. The input image and mask are partitioned into spatial patches and processed through multi-head self-attention layers. By computing global pairwise attention between unmasked context tokens and masked hole tokens at every layer, ViTInpainter models long-range semantic and structural relationships across the entire scene without relying on iterative boundary propagation.

2 Motivation

The classical OpenCV Telea baseline (`cv2.INPAINT_TELEA`) fills missing regions by propagating colour and texture gradients inward from the 1-pixel mask boundary using the fast marching method. While computationally lightweight ($\sim$ 0.014–0.025 s per image), Telea has two fundamental limitations:

- **Foreground bleeding on tight masks:** When evaluated with `sam_native`, residual foreground pixels and cast shadows right outside the object mask are propagated inward, creating colour smears.
- **Lack of semantic structure on large masks:** When masks are dilated or converted into coarse shapes (`sam_bbox`, `sam_ellipse`), local boundary interpolation blurs high-frequency textures and breaks geometric edges.

By introducing **LaMa** and **ViTInpainter**, we test whether global receptive fields—achieved via frequency-domain convolutions in LaMa and patch-level self-attention in ViTInpainter—can synthesize coherent background textures and significantly improve both pixel-level reconstruction (PSNR↑, SSIM↑) and perceptual quality (LPIPS↓) across varying mask geometries on CORNE-`Val`.

3 Implementation and Differences from Baseline

3.1 Resolving the Week 2 LaMa Blurring Issue

In Week 2, our initial `big-lama` runs produced smeared, blurred object silhouettes rather than removing the object. Following the diagnostic hypotheses formulated last week, we identified and resolved the issue in the LaMa wrapper:

- **Tensor conditioning and mask convention:** The TorchScript `big-lama` model expects an RGB float tensor in $[0, 1]$ together with a strictly binary mask in $\{0, 1\}$ ($1 =$ hole), where the masked region of the input image is explicitly zeroed out (image $\times$ (1 $-$ mask)) prior to passing through the FFC generator.
- **Padding and cropping:** Because CORNE-Val images keep their native high resolution (up to $\sim$ 1500 px), inputs are padded to a multiple of 8 pixels using reflect padding for the image and zero padding for the mask (preventing padded borders from being treated as holes), then cropped back to the exact original dimensions after inference.

3.2 Main Differences Compared with the Baseline

All three methods implement the unified `inpaint(image, mask)` interface in `benchmark.py` and are evaluated zero-shot without fine-tuning on the 218 samples of `CORNE-Val`. Table 1 summarizes the architectural and implementation differences compared to OpenCV Telea.

Property	OpenCV Telea (Baseline)	Pretrained LaMa	ViTInpainter
Paradigm	Classical PDE / Fast Marching	CNN + Fast Fourier Conv.	Vision Transformer (Self-Attention)
Pretrained weights	None (non-learned)	`big-lama` (TorchScript)	Pretrained ViT checkpoint
Receptive field	Local boundary neighbourhood	Global (frequency domain)	Global (all-to-all patch attention)
Preprocessing	RGB $\rightarrow$ BGR `uint8`	Float $[0, 1]$, reflect pad $\times 8$	Float tensor, patch tokenization
Compute device	CPU	GPU (CUDA / MPS)	GPU (CUDA / MPS)
Avg. inference time	$\sim$ 0.014–0.025 s	$\sim$ 0.63–0.64 s	$\sim$ 0.28–0.32 s

Table 1: Summary of implementation differences between the baseline and the two new models.

4 Initial Results and Comparison with Baseline

4.1 Quantitative Results on CORNE-Val

Tables 2 and 3 report the quantitative results (mean $\pm$ std over 218 samples of `CORNE-Val`) for pretrained LaMa and ViTInpainter across all six mask experiments, including both masked-region metrics (primary) and full-image metrics (secondary check). Table 4 compares region metrics directly against the Week 2 OpenCV Telea baseline.

4.2 Qualitative Results and Discussion

Key Observations

- **LaMa achieves the strongest overall reconstruction:** Across every mask configuration, LaMa delivers the best PSNR_{reg} and $\text{LPIPS}_{\text{reg}}$. On `eff_native`, LaMa reaches 19.75 dB PSNR (+2.33 dB over Telea) and lowers region LPIPS from 0.420 to 0.328 (-21.9%). Even on coarse bounding-box masks (`sam_bbox`), LaMa maintains 17.26 dB PSNR vs. 14.45 dB for Telea.

Mask	$\text{PSNR}_{\text{reg}} \uparrow$	$\text{SSIM}_{\text{reg}} \uparrow$	$\text{LPIPS}_{\text{reg}} \downarrow$	$\text{PSNR}_{\text{full}} \uparrow$	$\text{SSIM}_{\text{full}} \uparrow$	$\text{LPIPS}_{\text{full}} \downarrow$	Tim
sam_native	15.94 ± 4.96	0.657 ± 0.172	0.359 ± 0.100	27.21 ± 5.34	0.936 ± 0.039	0.086 ± 0.036	0.634 ±
sam_dilated10	16.83 ± 5.34	0.670 ± 0.177	0.357 ± 0.096	27.66 ± 5.56	0.935 ± 0.041	0.088 ± 0.037	0.641 ±
sam_dilated20	17.53 ± 5.79	0.672 ± 0.181	0.358 ± 0.092	27.84 ± 5.69	0.932 ± 0.042	0.091 ± 0.038	0.626 ±
sam_bbox	17.26 ± 5.13	0.588 ± 0.226	0.456 ± 0.110	27.40 ± 5.63	0.925 ± 0.053	0.101 ± 0.055	0.636 ±
sam_ellipse	14.94 ± 4.46	0.545 ± 0.198	0.483 ± 0.092	25.66 ± 5.02	0.925 ± 0.049	0.100 ± 0.050	0.645 ±
eff_native	$\mathbf{19.75 \pm 6.06}$	$\mathbf{0.711 \pm 0.180}$	$\mathbf{0.328 \pm 0.090}$	$\mathbf{29.95 \pm 4.97}$	$\mathbf{0.936 \pm 0.041}$	$\mathbf{0.088 \pm 0.037}$	0.642 ±

Table 2: Pretrained LaMa region and full-image metrics (mean ± std over CORNE-Val, 218 samples).

Mask	$\text{PSNR}_{\text{reg}} \uparrow$	$\text{SSIM}_{\text{reg}} \uparrow$	$\text{LPIPS}_{\text{reg}} \downarrow$	$\text{PSNR}_{\text{full}} \uparrow$	$\text{SSIM}_{\text{full}} \uparrow$	$\text{LPIPS}_{\text{full}} \downarrow$	Tim
sam_native	11.76 ± 3.89	0.503 ± 0.160	0.489 ± 0.123	24.86 ± 5.06	0.924 ± 0.042	0.097 ± 0.042	0.285 ±
sam_dilated10	14.66 ± 4.44	0.641 ± 0.154	0.383 ± 0.099	26.49 ± 5.21	0.933 ± 0.040	0.090 ± 0.037	0.315 ±
sam_dilated20	15.67 ± 5.17	0.649 ± 0.160	0.381 ± 0.093	26.78 ± 5.49	0.930 ± 0.042	0.092 ± 0.038	0.301 ±
sam_bbox	14.20 ± 4.28	0.466 ± 0.202	0.534 ± 0.090	25.27 ± 5.31	0.918 ± 0.058	0.105 ± 0.055	0.299 ±
sam_ellipse	11.34 ± 3.51	0.404 ± 0.153	0.581 ± 0.088	23.27 ± 4.74	0.915 ± 0.052	0.106 ± 0.051	0.298 ±
eff_native	$\mathbf{18.96 \pm 5.61}$	$\mathbf{0.713 \pm 0.162}$	$\mathbf{0.332 \pm 0.088}$	$\mathbf{29.69 \pm 5.08}$	$\mathbf{0.936 \pm 0.041}$	$\mathbf{0.088 \pm 0.038}$	**0.275** ±

Table 3: ViTInpainter region and full-image metrics (mean ± std over CORNE-Val, 218 samples).

Mask	Model	$\text{PSNR}_{\text{reg}} \uparrow$	$\text{SSIM}_{\text{reg}} \uparrow$	$\text{LPIPS}_{\text{reg}} \downarrow$	Time, s↓
sam_native	OpenCV Telea (Baseline)	12.74 ± 4.01	0.603 ± 0.146	0.487 ± 0.111	$\mathbf{0.014 \pm 0.009}$
	ViTInpainter	11.76 ± 3.89	0.503 ± 0.160	0.489 ± 0.123	0.285 ± 0.172
	Pretrained LaMa	$\mathbf{15.94 \pm 4.96}$	$\mathbf{0.657 \pm 0.172}$	$\mathbf{0.359 \pm 0.100}$	0.634 ± 0.201
sam_dilated10	OpenCV Telea (Baseline)	14.65 ± 4.48	0.664 ± 0.147	0.446 ± 0.099	$\mathbf{0.018 \pm 0.012}$
	ViTInpainter	14.66 ± 4.44	0.641 ± 0.154	0.383 ± 0.099	0.315 ± 0.179
	Pretrained LaMa	$\mathbf{16.83 \pm 5.34}$	$\mathbf{0.670 \pm 0.177}$	$\mathbf{0.357 \pm 0.096}$	0.641 ± 0.251
sam_dilated20	OpenCV Telea (Baseline)	15.26 ± 4.75	0.671 ± 0.149	0.453 ± 0.095	$\mathbf{0.021 \pm 0.013}$
	ViTInpainter	15.67 ± 5.17	0.649 ± 0.160	0.381 ± 0.093	0.301 ± 0.172
	Pretrained LaMa	$\mathbf{17.53 \pm 5.79}$	$\mathbf{0.672 \pm 0.181}$	$\mathbf{0.358 \pm 0.092}$	0.626 ± 0.157
sam_bbox	OpenCV Telea (Baseline)	14.45 ± 4.09	0.545 ± 0.196	0.608 ± 0.091	$\mathbf{0.025 \pm 0.021}$
	ViTInpainter	14.20 ± 4.28	0.466 ± 0.202	0.534 ± 0.090	0.299 ± 0.181
	Pretrained LaMa	$\mathbf{17.26 \pm 5.13}$	$\mathbf{0.588 \pm 0.226}$	$\mathbf{0.456 \pm 0.110}$	0.636 ± 0.156
sam_ellipse	OpenCV Telea (Baseline)	12.49 ± 3.57	0.506 ± 0.162	0.605 ± 0.077	$\mathbf{0.022 \pm 0.019}$
	ViTInpainter	11.34 ± 3.51	0.404 ± 0.153	0.581 ± 0.088	0.298 ± 0.179
	Pretrained LaMa	$\mathbf{14.94 \pm 4.46}$	$\mathbf{0.545 \pm 0.198}$	$\mathbf{0.483 \pm 0.092}$	0.645 ± 0.174
eff_native	OpenCV Telea (Baseline)	17.42 ± 5.40	$\mathbf{0.722 \pm 0.146}$	0.420 ± 0.099	$\mathbf{0.023 \pm 0.016}$
	ViTInpainter	18.96 ± 5.61	0.713 ± 0.162	0.332 ± 0.088	0.275 ± 0.169
	Pretrained LaMa	$\mathbf{19.75 \pm 6.06}$	0.711 ± 0.180	$\mathbf{0.328 \pm 0.090}$	0.642 ± 0.170

Table 4: Head-to-head comparison of region metrics (mean ± std) between OpenCV Telea, ViTInpainter, and Pretrained LaMa across all six mask experiments.

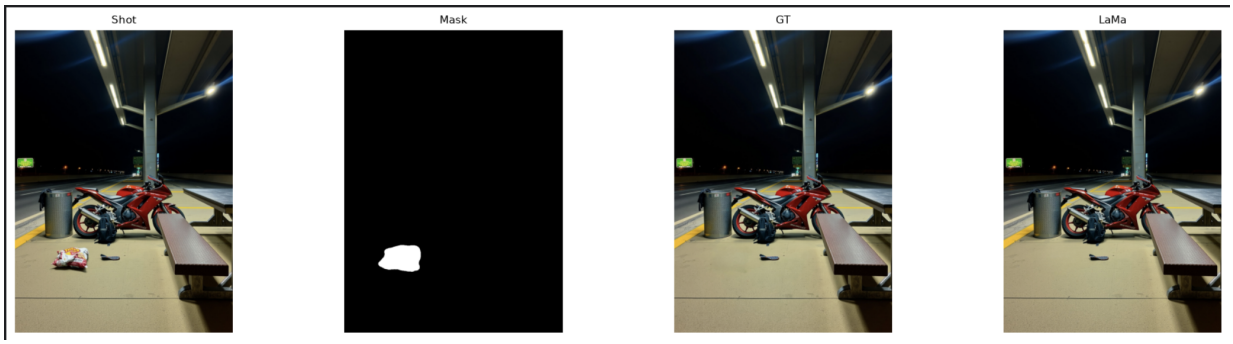


Figure 1: Example of pretrained LaMa output after fixing the input conditioning: the foreground object is completely removed and replaced with sharp, continuous pavement texture matching the ground truth.

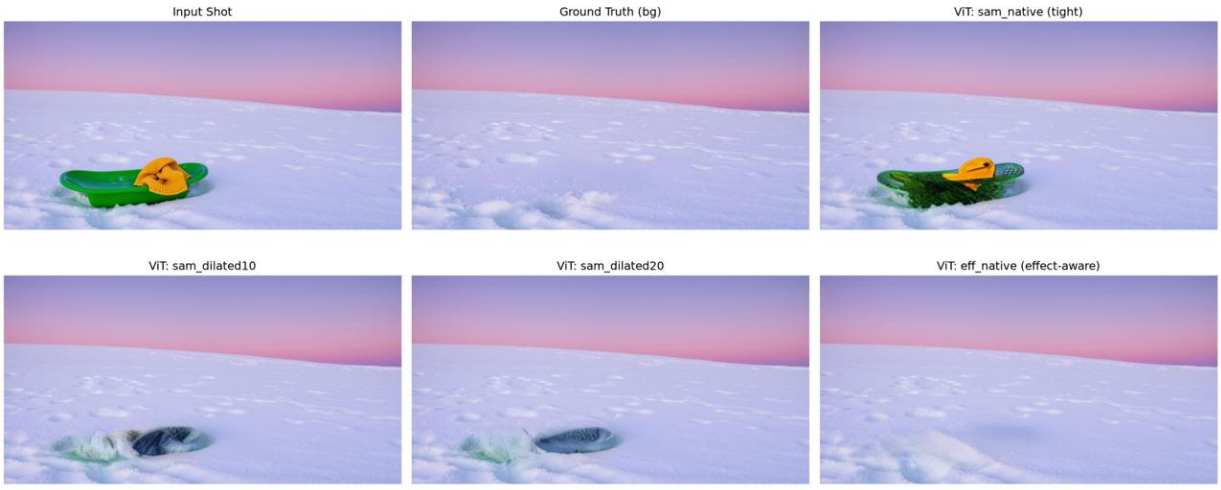


Figure 2: Qualitative comparison of ViTInpainter across mask configurations: `sam_native` (tight mask) hallucinates object remnants due to unmasked boundary pixels and shadows; `sam_dilated10` and `sam_dilated20` reduce colour bleeding but still hallucinate dark structures along the shadow boundary; `eff_native` (covering both object and shadows) achieves clean snow reconstruction.

- **ViTInpainter offers a strong speed–quality trade-off on clean masks:** When provided with the effect-aware mask (`eff_native`), ViTInpainter reaches 18.96 dB PSNR and 0.332 LPIPS—nearly matching LaMa while running more than 2× faster (0.275 s vs. 0.642 s per image).
- **Effect-aware masks (`eff_native`) are critical across all paradigms:** Removing cast shadows and reflections alongside the object core yields the highest PSNR, SSIM, and LPIPS for Telea, ViTInpainter, and LaMa alike.

5 Failure Analysis and Problems Observed

In accordance with the project guidelines, we inspected representative failure cases across the evaluated models and identified three systematic failure modes along with their underlying causes, as well as an infrastructure bottleneck encountered during our initial `FLUX.1-Fill` tests.

5.1 Representative Failure Cases

- **Failure Case 1 — Hallucinations along shadow boundaries and tight mask edges (Both ViTInpainter and LaMa):** When using the tight object mask (`sam_native`) or moderately dilated masks (`sam_dilated10`, `sam_dilated20`) that do not fully cover the cast shadow, both ViTInpainter and LaMa hallucinate dark blobs, pseudo-shadows, or distorted object remnants inside the inpainted region. As shown in Figure 2, ViTInpainter attends to unmasked green/yellow boundary pixels on `sam_native` (11.76 dB PSNR) and synthesizes a dark grey object on `sam_dilated10/20` where the shadow edge touches the mask border. Similarly, LaMa's PSNR drops by 3.81 dB on `sam_native` (15.94 dB) and 2.22 dB on `sam_dilated20` (17.53 dB) compared to `eff_native` (19.75 dB), because its Fast Fourier Convolutions treat the unmasked dark shadow boundary as a legitimate structural cue and propagate a synthetic object/shadow continuation into the hole.
- **Failure Case 2 — Structural degradation and over-smoothing on coarse geometric masks (`sam_bbox` and `sam_ellipse`):** Replacing the contour mask with an enclosing ellipse or bounding box removes a large area of valid background context around the object. On `sam_ellipse`, region LPIPS degrades sharply across all three models: from 0.328 to 0.483 in LaMa, from 0.332 to 0.581 in ViTInpainter, and from 0.420 to 0.605 in Telea. Without nearby structural anchors, non-generative feed-forward networks produce low-frequency blurry patches across the centre of large holes.
- **Failure Case 3 — High-frequency texture mismatch on non-periodic backgrounds:** While LaMa excels on periodic structures (such as tiled pavement or road lines in Figure 1) due to its FFT branch, both LaMa and ViTInpainter struggle when the removed object occludes complex, non-repetitive 3D geometry (e.g., clutter, foliage, or intersecting furniture edges at 1500 px resolution), resulting in subtle seam artifacts or repetitive spectral ringing.

5.2 Resource Bottleneck with `FLUX.1-Fill`

- **Underestimated inference time and VRAM requirements for `FLUX.1-Fill`:** We originally planned to complete the evaluation of the diffusion/rectified-flow model `FLUX.1-Fill`

during Week 3 alongside LaMa and ViTInpainter. However, we underestimated its computational footprint: as a 12B-parameter flow-matching transformer requiring multi-step iterative denoising at high resolution, running `FLUX.1-Fill` across all $218 \times 6 = 1{,}308$ benchmark combinations exceeded our available GPU memory and compute time budget for this week. We are currently optimizing the pipeline (using `bfloat16` quantization, CPU offloading, and batched scheduling) to complete the `FLUX.1-Fill` experiments in Week 4.

6 Next Steps

Plan for week 4

- **Complete `FLUX.1-Fill` benchmark evaluation:** Allocate sufficient GPU compute time and use quantized/offloaded inference to run `FLUX.1-Fill` across the `CORNE-Val` mask experiments.
- **Cross-paradigm comparison:** Compare classical (Telea), CNN/FFC (LaMa), Vision Transformer (ViTInpainter), and Diffusion (`FLUX.1-Fill`) models in terms of reconstruction fidelity, perceptual realism on coarse masks, and inference latency.
- **Finalize failure analysis and deliverables:** Add visual failure examples for LaMa shadow hallucinations and `FLUX.1-Fill` to our failure analysis, and prepare the final two-page technical summary and live demo slides.

7 Code Repository

The current project code is available at:
https://github.com/Kazualov/bye-bye-AI